

□ Acquisition des données —Excel avec Python

Le support est une propriété de **Bechir Bejaoui**

□ Préparation

Avant de commencer, générer tous les fichiers Excel de l'atelier en une seule commande :

Prérequis

```
pip install openpyxl pandas
```

Générer tous les fichiers d'entrée

```
python generate_excel.py
```

Les fichiers sont créés dans le dossier data/

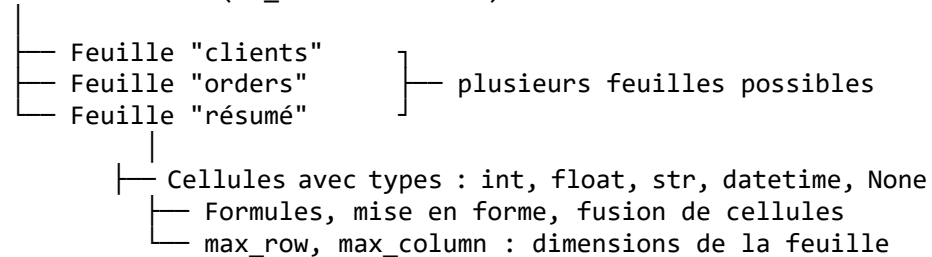
q01_ids.xlsx, q02_maxrow.xlsx, q03_big.xlsx ... q20_pipeline.xlsx

Pas besoin de créer les fichiers à la main —generate_excel.py les produit tous automatiquement dans data/.

1 □ Introduction —Python et les fichiers Excel

Un fichier Excel (.xlsx) n'est pas un simple fichier texte comme le CSV. C'est une **archive ZIP** contenant du XML. Python ne peut pas le lire directement avec open() —il faut une bibliothèque dédiée.

Fichier Excel (FL_insurance.xlsx)



Deux approches complémentaires :

	openpyxl	pandas
Accès	Cellule par cellule	Colonne entière / DataFrame

	openpyxl	pandas
Types	Respecte le type Excel	Infère automatiquement
Mémoire	Charge tout (ou read_only)	Charge tout
Usage	Lecture fine, écriture, format	Analyse, stats, jointures
Installation	<code>pip install openpyxl</code>	<code>pip install pandas</code> <code>openpyxl</code>

- **openpyxl** : accès direct aux cellules —`ws["A1"].value`, `ws.cell(row=1, column=1)` —idéal pour inspecter la structure
- **pandas** : transforme une feuille entière en DataFrame —idéal pour filtrer, trier, agréger
- **read_only=True** : mode streaming d'openpyxl —pour les gros fichiers, évite de tout charger en RAM

2 □ Lecture d'une cellule avec openpyxl

Fichier utilisé : `data/q01_ids.xlsx`

Contenu généré par `generate_excel.py` :

Sheet1

```

A1 : policyID    ← en-tête (str)
A2 : 119736      ← entier
A3 : 448094      ← entier

from pathlib import Path
from openpyxl import load_workbook

xlsx_path = Path("data") / "q01_ids.xlsx"

# Chargement du classeur complet
wb = load_workbook(xlsx_path)

# Feuille active (la première par défaut)
ws = wb.active
print("Nom de la feuille :", ws.title)    # Sheet1

# — Accès à une cellule —————

# Méthode 1 : notation Lettre-chiffre (comme Excel)
cellule_a1 = ws["A1"]
print("A1 :", cellule_a1.value)           # policyID
print("Type :", type(cellule_a1.value))   # <class 'str'>

```

```
# Méthode 2 : row/column (indexés à partir de 1, pas 0)
cellule_a2 = ws.cell(row=2, column=1)
print("A2 :", cellule_a2.value)          # 119736
print("Type :", type(cellule_a2.value))  # <class 'int'>

cellule_a3 = ws.cell(row=3, column=1)
print("A3 :", cellule_a3.value)          # 448094

Nom de la feuille : Sheet1
A1 : policyID
Type : <class 'str'>
A2 : 119736
Type : <class 'int'>
A3 : 448094
```

- **load_workbook(path)** : charge le fichier .xlsx entier en mémoire — retourne un objet Workbook
- **wb.active** : retourne la feuille active (celle sélectionnée à la sauvegarde) —ici Sheet1
- **ws["A1"].value** : notation Excel classique —la propriété .value extrait le contenu Python
- **ws.cell(row=2, column=1)** : indexation numérique —**commence à 1**, pas à 0 comme les listes Python
- **openpyxl préserve les types** : 119736 est stocké comme entier dans Excel → Python le reçoit comme int

3 □ Dimensions et max_row —le piège des cellules fantômes

Fichier utilisé : data/q02_maxrow.xlsx

Contenu généré par generate_excel.py :

```
A1 : "col1"    ← en-tête
A2 : "ok"      ← donnée
A100: None     ← cellule vidée (était "temp", remplacée par None)
               ← mais la trace reste dans le fichier XML
```

```
from pathlib import Path
from openpyxl import load_workbook
```

```
xlsx_path = Path("data") / "q02_maxrow.xlsx"
wb = load_workbook(xlsx_path)
ws = wb.active
```

```

print("=== max_row et max_column bruts ===")
print("max_row      :", ws.max_row)      # peut retourner 100, pas 2 !
print("max_column  :", ws.max_column)    # 1

print("\n=== Contenu réel des cellules ===")
for row in ws.iter_rows():
    for cell in row:
        if cell.value is not None:
            print(f" Cellule {cell.coordinate} : {cell.value!r}")

print("\n=== Comptage des lignes réellement non vides ===")
lignes_reelles = 0
for row in ws.iter_rows():
    if any(cell.value is not None for cell in row):
        lignes_reelles += 1

print("Lignes non vides :", lignes_reelles) # 2 (A1 et A2)
print("max_row          :", ws.max_row)     # 100 — attention !

=== max_row et max_column bruts ===
max_row      : 100
max_column   : 1

=== Contenu réel des cellules ===
Cellule A1 : 'col1'
Cellule A2 : 'ok'

=== Comptage des lignes réellement non vides ===
Lignes non vides : 2
max_row          : 100

Pourquoi max_row = 100 alors qu'il y a 2 lignes ?

ws["A100"] = "temp"    # ← écrit en A100
ws["A100"] = None      # ← la cellule est "vidée"
                      # mais Excel mémorise qu'elle a été touchée
                      # max_row reste à 100 dans le XML

```

Solution : compter avec `any(cell.value is not None for cell in row)`

- **ws.max_row** : dimension mémorisée par Excel —peut surestimer si des cellules ont été écrites puis effacées
- **ws.max_column** : même problème pour les colonnes
- **ws.iter_rows()** : parcourt toutes les lignes jusqu'à `max_row` —à combiner avec un test `is not None`
- **règle pratique** : toujours compter les lignes réelles manuellement plutôt

que de faire confiance à max_row

4 □ Lecture d'un gros fichier —mode read_only

Fichier utilisé : data/q03_big.xlsx et data/q04_readonly.xlsx

Contenu généré par generate_excel.py :

q03_big.xlsx : 5000 lignes × 10 colonnes (h1..h10), valeurs numériques
q04_readonly.xlsx : 30000 lignes × 3 colonnes (id, statecode, amount)

```
from pathlib import Path
import time
from openpyxl import load_workbook
```

```
xlsx_path = Path("data") / "q04_readonly.xlsx"
```

```
# — Mode normal : charge tout en RAM —————
t0 = time.time()
wb_normal = load_workbook(xlsx_path)
ws = wb_normal.active
lignes = sum(1 for row in ws.iter_rows() if any(c.value for c in row))
t1 = time.time()
print(f"Mode normal : {lignes} lignes lues en {t1-t0:.2f}s")
wb_normal.close()
```

```
# — Mode read_only : streaming, bien moins gourmand en RAM —————
t2 = time.time()
wb_ro = load_workbook(xlsx_path, read_only=True)
ws_ro = wb_ro.active
compteur = 0
for row in ws_ro.iter_rows():
    if any(cell.value is not None for cell in row):
        compteur += 1
t3 = time.time()
print(f"Mode read_only: {compteur} lignes lues en {t3-t2:.2f}s")
wb_ro.close()
```

```
# — Accès aux 3 premières lignes du gros fichier —————
wb = load_workbook(Path("data") / "q03_big.xlsx", read_only=True)
ws = wb.active
print("\nAperçu q03_big.xlsx (3 premières lignes) :")
for i, row in enumerate(ws.iter_rows(max_row=3, values_only=True)):
    print(f" ligne {i+1} : {row}")
wb.close()
```

Mode normal : 30001 lignes lues en 1.84s
Mode read_only: 30001 lignes lues en 0.63s

Aperçu q03_big.xlsx (3 premières lignes) :
ligne 1 : ('h1', 'h2', 'h3', 'h4', 'h5', 'h6', 'h7', 'h8', 'h9', 'h10')
ligne 2 : (201, 202, 203, 204, 205, 206, 207, 208, 209, 210)
ligne 3 : (301, 302, 303, 304, 305, 306, 307, 308, 309, 310)

- **load_workbook(path, read_only=True)** : mode streaming —les lignes sont lues une par une sans tout charger en RAM —**indispensable sur les gros fichiers**
 - **values_only=True** dans iter_rows : retourne directement les valeurs (pas les objets Cell) —plus simple et plus rapide
 - **max_row=3** : limite la lecture aux 3 premières lignes —utile pour un aperçu rapide
 - **toujours wb.close()** en mode read_only : le fichier reste ouvert jusqu' à fermeture explicite
-

5 □ Types et valeurs manquantes avec openpyxl

Fichier utilisé : data/q05_dtype.xlsx

Contenu généré par generate_excel.py :

A	B
policyID	statecode
119736	FL
None	FL ← policyID manquant (None dans Excel)
448094	TX

```
from pathlib import Path
from openpyxl import load_workbook

xlsx_path = Path("data") / "q05_dtype.xlsx"
wb = load_workbook(xlsx_path)
ws = wb.active

print(f"{'Ligne':<6} {'policyID':<15} {'Type':<15} {'statecode'}")
print("-" * 50)

for i, row in enumerate(ws.iter_rows(min_row=2, values_only=True)):
    policy_id, statecode = row[0], row[1]
    type_id = type(policy_id).__name__

    # Détecter et signaler les valeurs manquantes
```

```

statut = "?" if policy_id is not None else "MANQUANT"

print(f" {i+2:<4} {str(policy_id):<15} {type_id:<15} {statecode} {statut}")

# Compter Les valeurs manquantes
nb_manquants = sum(
    1 for row in ws.iter_rows(min_row=2, values_only=True)
    if row[0] is None
)
print(f"\npolicyID manquants : {nb_manquants}")

```

Ligne	policyID	Type	statecode
2	119736	int	FL ?
3	None	NoneType	FL ? MANQUANT
4	448094	int	TX ?

policyID manquants : 1

- **None en openpyxl** : une cellule vide Excel retourne None (pas "", pas 0)
—toujours tester avec `is None`
- **min_row=2** dans `iter_rows` : commence à lire à partir de la ligne 2 —
saute l'en-tête
- **values_only=True** : retourne les valeurs directement —`row[0]`, `row[1]`
au lieu de `row[0].value`
- **les types sont préservés** : 119736 reste `int`, une date serait un `datetime`, une cellule vide est `None`

6 Lire seulement certaines colonnes —`usecols` avec pandas

Fichier utilisé : `data/q06_usecols.xlsx`

Contenu généré par `generate_excel.py` :

20 colonnes : `c1`, `c2`, `c3`, ... `c20`

10000 lignes de données numériques (`r * i`)

```

from pathlib import Path
import pandas as pd
import time

```

```

xlsx_path = Path("data") / "q06_usecols.xlsx"

```

— Lire toutes les colonnes

```

t0 = time.time()
df_all = pd.read_excel(xlsx_path)
t1 = time.time()
print(f"Toutes colonnes : {df_all.shape} en {t1-t0:.2f}s")
print(f" Colonnes : {list(df_all.columns)[:5]} ... ({len(df_all.columns)} total)")

# — Lire seulement 2 colonnes —————
t2 = time.time()
df_sub = pd.read_excel(xlsx_path, usecols=["c1", "c2"])
t3 = time.time()
print(f"\nUsecols [c1, c2] : {df_sub.shape} en {t3-t2:.2f}s")
print(df_sub.head(3))

# — Sélection par position (0-indexé) —————
df_pos = pd.read_excel(xlsx_path, usecols=[0, 9]) # c1 et c10
print(f"\nUsecols [0, 9] (c1 et c10) :")
print(df_pos.head(3))

Toutes colonnes : (10000, 20) en 3.21s
Colonnes : ['c1', 'c2', 'c3', 'c4', 'c5'] ... (20 total)

Usecols [c1, c2] : (10000, 2) en 1.18s
   c1  c2
0  101 102
1  201 202
2  301 302

Usecols [0, 9] (c1 et c10) :
   c1  c10
0  101  110
1  201  210
2  301  310

```

- **usecols=["c1", "c2"]** : ne charge que les colonnes listées —gain de temps et de mémoire significatif sur les fichiers larges
- **usecols=[0, 9]** : sélection par position 0-indexée —pratique quand on ne connaît pas les noms de colonnes à l'avance
- **règle de performance** : toujours utiliser `usecols` sur les fichiers avec beaucoup de colonnes —inutile de charger 20 colonnes si on n'en utilise que 2
- **df.shape** : retourne (nb_lignes, nb_colonnes) —à vérifier systématiquement après lecture

7 □ Limiter les lignes lues —nrows

Fichier utilisé : data/q07_nrows.xlsx

Contenu généré par generate_excel.py :

2 colonnes : id, value

50 000 lignes (id de 1 à 50000, value = "v1", "v2", ...)

```
from pathlib import Path
import pandas as pd
import time
```

```
xlsx_path = Path("data") / "q07_nrows.xlsx"
```

```
# — Aperçu rapide : lire seulement 5 lignes —————
```

```
df_aperçu = pd.read_excel(xlsx_path, nrows=5)
print("Aperçu (5 lignes) :")
print(df_aperçu)
```

```
# — Comparaison temps de lecture —————
```

```
t0 = time.time()
df_100 = pd.read_excel(xlsx_path, nrows=100)
t1 = time.time()
```

```
t2 = time.time()
df_all = pd.read_excel(xlsx_path)
t3 = time.time()
```

```
print(f"\nnrows=100 : {df_100.shape[0]} lignes en {t1-t0:.2f}s")
print(f"nrows=None : {df_all.shape[0]} lignes en {t3-t2:.2f}s")
```

```
# — Cas pratique : valider le schéma sans charger 50 000 lignes —————
```

```
COLONNES_REQUISES = {"id", "value"}
df_test = pd.read_excel(xlsx_path, nrows=1) # une seule ligne suffit pour voir le head
colonnes_presentes = set(df_test.columns)
manquantes = COLONNES_REQUISES - colonnes_presentes
```

```
if manquantes:
    raise ValueError(f"Colonnes manquantes : {manquantes}")
else:
    print(f"\n✅ Schéma valide : {sorted(colonnes_presentes)}")
    print(f"    Fichier complet : {df_all.shape[0]} lignes")
```

Aperçu (5 lignes) :

	id	value
0	1	v1

```
1  2  v2
2  3  v3
3  4  v4
4  5  v5
```

```
nrows=100 : 100 lignes en 0.31s
nrows=None : 50000 lignes en 8.74s
```

```
❑ Schéma valide : ['id', 'value']
   Fichier complet : 50000 lignes
```

- **nrows=5** : lit seulement les 5 premières lignes de données (hors en-tête) —parfait pour un aperçu rapide
- **nrows=1** pour valider le schéma : inutile de charger 50 000 lignes pour vérifier que les colonnes existent
- **gain de temps** : sur ce fichier, **nrows=100** est **28x plus rapide** que tout charger
- **combinaison nrows + usecols** : les deux options ensemble = lecture minimale —idéal pour les pipelines de validation

8 ❑ Détecter les doublons

Fichier utilisé : data/q08_dups.xlsx

Contenu généré par generate_excel.py :

```
policyID  statecode
100       FL
101       TX
100       FL    ← doublon exact de la ligne 1
102       TX
101       TX    ← doublon partiel (même policyID, même state)
```

```
from pathlib import Path
import pandas as pd
```

```
xlsx_path = Path("data") / "q08_dups.xlsx"
df = pd.read_excel(xlsx_path)
```

```
print("Données brutes :")
print(df.to_string(index=True))
```

```
# — Détecter les doublons complets (toutes colonnes identiques) —————
masque_doublons = df.duplicated()
print(f"\nDoublons complets : {masque_doublons.sum()}")
```

```

print(df[masque_doublons])

# — Détecter Les doublons sur policyID uniquement —————
masque_policy = df.duplicated(subset=["policyID"])
print(f"\npolicyID dupliqués : {masque_policy.sum()}")
print(df[masque_policy])

# — Supprimer Les doublons — garder La première occurrence —————
df_clean = df.drop_duplicates()
print(f"\nAvant : {len(df)} lignes | Après suppression : {len(df_clean)} lignes")
print(df_clean.to_string(index=False))

# — Supprimer Les doublons sur policyID uniquement —————
df_unique_policy = df.drop_duplicates(subset=["policyID"], keep="first")
print(f"\nSur policyID uniquement : {len(df_unique_policy)} lignes")

```

Données brutes :

	policyID	statecode
0	100	FL
1	101	TX
2	100	FL
3	102	TX
4	101	TX

Doublons complets : 1

	policyID	statecode
2	100	FL

policyID dupliqués : 2

	policyID	statecode
2	100	FL
4	101	TX

Avant : 5 lignes | Après suppression : 4 lignes

	policyID	statecode
	100	FL
	101	TX
	102	TX

Sur policyID uniquement : 3 lignes

- **df.duplicated()** : retourne un masque booléen — True pour chaque ligne qui est le doublon d'une ligne précédente
- **df[masque]** : filtrage par masque — affiche uniquement les lignes marquées True
- **subset=["policyID"]** : recherche les doublons sur une seule colonne — deux lignes peuvent avoir le même policyID avec des valeurs différentes

ailleurs

- **keep="first"** : garde la première occurrence, supprime les suivantes
—keep="last" pour l'inverse, keep=False pour supprimer toutes les occurrences

9 □ Lignes vides et valeurs manquantes

Fichier utilisé : data/q09_emptyrows.xlsx

Contenu généré par generate_excel.py :

a	b	c	
1	2	3	
None	None	None	← ligne entièrement vide
4	None	6	← ligne partiellement vide

```
from pathlib import Path
import pandas as pd
```

```
xlsx_path = Path("data") / "q09_emptyrows.xlsx"
df = pd.read_excel(xlsx_path)
```

```
print("Données brutes :")
print(df)
print(f"\nShape : {df.shape}")
```

```
# — Cartographie des valeurs manquantes —————
print("\nValeurs manquantes par colonne :")
print(df.isna().sum())
```

```
print("\nMasque complet des NaN :")
print(df.isna())
```

```
# — Supprimer les lignes entièrement vides —————
df_sans_vides = df.dropna(how="all")
print(f"\nAprès dropna(how='all') : {len(df_sans_vides)} lignes")
print(df_sans_vides)
```

```
# — Supprimer les lignes avec au moins un NaN —————
df_complet = df.dropna(how="any")
print(f"\nAprès dropna(how='any') : {len(df_complet)} lignes")
print(df_complet)
```

```
# — Remplir les NaN par une valeur par défaut —————
df_rempli = df.fillna(0)
```

```
print(f"\nAprès fillna(0) :")
print(df_rempli)
```

Données brutes :

```
   a    b  c
0  1.0  2.0  3
1  NaN  NaN NaN
2  4.0  NaN  6
```

Shape : (3, 3)

Valeurs manquantes par colonne :

```
a    1
b    2
c    1
dtype: int64
```

Après dropna(how='all') : 2 lignes

```
   a    b  c
0  1.0  2.0  3
2  4.0  NaN  6
```

Après dropna(how='any') : 1 lignes

```
   a    b  c
0  1.0  2.0  3
```

Après fillna(0) :

```
   a    b  c
0  1.0  2.0  3
1  0.0  0.0  0
2  4.0  0.0  6
```

Guide de traitement des NaN :

Ligne entièrement vide → dropna(how="all") supprime la ligne
 Ligne avec 1+ NaN → dropna(how="any") supprime la ligne
 Valeur NaN = 0 → fillna(0)
 Valeur NaN = moyenne → fillna(df["col"].mean())
 Garder NaN → ne rien faire, traiter cas par cas

- **NaN vs None** : pandas convertit les None Excel en NaN (Not a Number) pour les colonnes numériques —les deux se testent avec pd.isna()
- **how="all"** : supprime uniquement si **toutes** les colonnes sont NaN — préserve les lignes partiellement remplies
- **how="any"** : supprime dès qu'**une seule** colonne est NaN —plus strict

❑ Validation de schéma avant traitement

Fichier utilisé : data/q10_schema.xlsx

Contenu généré par generate_excel.py :

```
policyID  statecode
1         FL
```

← colonne "county" absente intentionnellement

```
from pathlib import Path
import pandas as pd
```

```
COLONNES_REQUISES = ["policyID", "statecode", "county"]
```

```
def valider_et_charger(chemin: Path) -> pd.DataFrame:
    """
    Charge un fichier Excel et vérifie Le schéma.
    Lève ValueError si des colonnes obligatoires manquent.
    """
    if not chemin.exists():
        raise FileNotFoundError(f"Fichier introuvable : {chemin}")

    # Lire une seule ligne pour inspecter le header sans tout charger
    df_header = pd.read_excel(chemin, nrows=0)
    colonnes_presentes = set(df_header.columns)
    manquantes = set(COLONNES_REQUISES) - colonnes_presentes

    if manquantes:
        raise ValueError(
            f"Schéma invalide — colonnes manquantes : {sorted(manquantes)}\n"
            f"  Colonnes présentes : {sorted(colonnes_presentes)}"
        )

    # Schéma valide → charger complètement
    return pd.read_excel(chemin)

# — Test avec fichier invalide —————
try:
    df = valider_et_charger(Path("data") / "q10_schema.xlsx")
    print("📄 Chargé :", df.shape)
except ValueError as e:
    print("❌", e)

# — Test avec fichier valide —————
try:
    df = valider_et_charger(Path("data") / "q08_dups.xlsx")
```

```

print("📄 Chargé :", df.shape)
except ValueError as e:
    print("📄", e)

```

📄 Schéma invalide — colonnes manquantes : ['county', 'statecode']
Colonnes présentes : ['policyID']

📄 Schéma invalide — colonnes manquantes : ['county']
Colonnes présentes : ['policyID', 'statecode']

- **nrows=0** : charge uniquement l'en-tête (0 lignes de données) — parfait pour inspecter le schéma sans charger les données
- **set(requises) - set(presentes)** : différence d'ensembles — retourne exactement les colonnes absentes
- **valider en premier** : si le schéma est invalide, on lève une exception claire avant tout traitement — évite un KeyError silencieux à la ligne 50 000

1 📄 📄 En-têtes sales et normalisation

Fichier utilisé : data/q11_dirty_headers.xlsx

Contenu généré par generate_excel.py :

```

" PolicyID "      "STATECODE"      "statecode"
   1              FL              FL

```

(espaces en début/fin, casse incohérente, doublon logique STATECODE/statecode)

```

from pathlib import Path
import pandas as pd

```

```

xlsx_path = Path("data") / "q11_dirty_headers.xlsx"
df = pd.read_excel(xlsx_path)

```

```

print("=== Colonnes brutes ===")
print(repr(list(df.columns)))
# [' PolicyID ', 'STATECODE', 'statecode']

```

```

# — Étape 1 : supprimer Les espaces en début/fin
df.columns = df.columns.str.strip()
print("\nAprès strip() :")
print(repr(list(df.columns)))
# ['PolicyID', 'STATECODE', 'statecode']

```

```

# — Étape 2 : tout en minuscules
df.columns = df.columns.str.lower()

```

```

print("\nAprès lower() :")
print(repr(list(df.columns)))
# ['policyid', 'statecode', 'statecode']

# — Étape 3 : détecter les doublons de colonnes —————
doublons_col = [col for col in df.columns if list(df.columns).count(col) > 1]
if doublons_col:
    print(f"\n🔍 Colonne(s) dupliquée(s) après normalisation : {set(doublons_col)}")
    # Renommer automatiquement
    cols = list(df.columns)
    seen = {}
    new_cols = []
    for col in cols:
        if col in seen:
            seen[col] += 1
            new_cols.append(f"{col}_{seen[col]}")
        else:
            seen[col] = 0
            new_cols.append(col)
    df.columns = new_cols
    print(f"Colonne(s) renommée(s) : {list(df.columns)}")

print("\nDataFrame final :")
print(df)

=== Colonnes brutes ===
['PolicyID ', 'STATECODE', 'statecode']

Après strip() :
['PolicyID', 'STATECODE', 'statecode']

Après lower() :
['policyid', 'statecode', 'statecode']

🔍 Colonne(s) dupliquée(s) après normalisation : {'statecode'}
Colonne(s) renommée(s) : ['policyid', 'statecode', 'statecode_1']

DataFrame final :
  policyid statecode statecode_1
0         1         FL         FL

• df.columns.str.strip() : vectorisé — applique .strip() sur tous les noms de colonnes d'un coup
• df.columns.str.lower() : normalise la casse — STATECODE et statecode deviennent identiques
• doublon après normalisation : deux colonnes peuvent avoir des noms différents en apparence (STATECODE/statecode) mais identiques après

```


normalisation —à détecter et renommer

1 □2 □ Dates et types mixtes

Fichier utilisé : data/q12_dates.xlsx

Contenu généré par generate_excel.py :

id	date_ok	date_text
1	datetime(2026,3,1)	"2026-03-01"
2	datetime(2026,3,2)	"03/02/2026"

```
from pathlib import Path
import pandas as pd
```

```
xlsx_path = Path("data") / "q12_dates.xlsx"
df = pd.read_excel(xlsx_path)
```

```
print("Types après lecture :")
print(df.dtypes)
print("\nAperçu :")
print(df)
```

```
# — date_ok : déjà un datetime (Excel → pandas Le préserve) —————
```

```
print(f"\ndate_ok type      : {type(df['date_ok'][0])}")
print(f"date_ok valeur    : {df['date_ok'][0]}")
```

```
# — date_text : Lue comme string — formats différents —————
```

```
print(f"\ndate_text type      : {type(df['date_text'][0])}")
print(f"date_text[0]       : {df['date_text'][0]!r}") # "2026-03-01"
print(f"date_text[1]       : {df['date_text'][1]!r}") # "03/02/2026"
```

```
# — Conversion de date_text en datetime —————
```

```
df["date_parsed"] = pd.to_datetime(df["date_text"], errors="coerce")
print("\nAprès to_datetime(errors='coerce') :")
print(df[["date_text", "date_parsed"]])
```

```
# Extraire l'année, le mois, le jour
```

```
df["annee"] = df["date_ok"].dt.year
df["mois"]  = df["date_ok"].dt.month
print("\nExtraction année/mois :")
print(df[["date_ok", "annee", "mois"]])
```

```
Types après lecture :
id                int64
```

```

date_ok      datetime64[ns]
date_text    object
dtype: object

date_ok type : <class 'pandas._libs.tslibs.timestamps.Timestamp'>
date_ok valeur : 2026-03-01 00:00:00

date_text type : <class 'str'>
date_text[0] : '2026-03-01'
date_text[1] : '03/02/2026'

```

```

Après to_datetime(errors='coerce') :
      date_text  date_parsed
0  2026-03-01    2026-03-01
1  03/02/2026    2026-03-02

```

```

Extraction année/mois :
      date_ok  annee  mois
0  2026-03-01    2026     3
1  2026-03-02    2026     3

```

- **datetime Excel** → **pandas** : une cellule Excel de type datetime est automatiquement convertie en Timestamp pandas —pas de conversion manuelle nécessaire
- **texte date** : une cellule texte "2026-03-01" reste un str —utiliser `pd.to_datetime()` pour convertir
- **errors="coerce"** : si la conversion échoue (format non reconnu), met NaT (Not a Time) au lieu de lever une exception
- **.dt.year, .dt.month** : accesseur vectorisé pour extraire des composants d'une colonne datetime

1 □3 □ Classeur multi-feuilles et jointure

Fichier utilisé : data/q14_sheets.xlsx

Contenu généré par `generate_excel.py` :

Feuille "clients" :		Feuille "orders" :		
client_id	name	order_id	client_id	amount
1	A	10	1	50
2	B	11	2	70

```

from pathlib import Path
import pandas as pd

```

```

xlsx_path = Path("data") / "q14_sheets.xlsx"

# — Inspecter les feuilles disponibles —————
xf = pd.ExcelFile(xlsx_path)
print("Feuilles disponibles :", xf.sheet_names)

# — Charger chaque feuille séparément —————
clients = pd.read_excel(xlsx_path, sheet_name="clients")
orders = pd.read_excel(xlsx_path, sheet_name="orders")

print("\nClients :")
print(clients)
print("\nOrders :")
print(orders)

# — Charger toutes les feuilles d'un coup —————
all_sheets = pd.read_excel(xlsx_path, sheet_name=None)
print("\nToutes les feuilles (dict) :", list(all_sheets.keys()))

# — Jointure : enrichir les commandes avec le nom du client —————
enriched = orders.merge(clients, on="client_id", how="left")
print("\nCommandes enrichies :")
print(enriched)

# Calculer le total par client
total_par_client = enriched.groupby("name")["amount"].sum()
print("\nTotal commandes par client :")
print(total_par_client)

Feuilles disponibles : ['clients', 'orders']

Clients :
   client_id name
0          1    A
1          2    B

Orders :
   order_id  client_id  amount
0         10          1      50
1         11          2      70

Toutes les feuilles (dict) : ['clients', 'orders']

Commandes enrichies :
   order_id  client_id  amount name
0         10          1      50    A

```

```
1      11      2      70      B
```

Total commandes par client :

```
name
```

```
A      50
```

```
B      70
```

- **pd.ExcelFile(path)** : ouvre le fichier sans charger les données — `.sheet_names` liste les feuilles disponibles
- **sheet_name=None** : charge **toutes** les feuilles dans un dictionnaire {nom_feuille: DataFrame}
- **orders.merge(clients, on="client_id", how="left")** : jointure SQL-like — `how="left"` garde toutes les commandes même si le client n'existe pas
- **groupby("name")["amount"].sum()** : agrégation par client — équivalent d'un `GROUP BY` SQL

1 □4 □ Pipeline complet —validation, nettoyage, export

Fichier utilisé : data/q20_pipeline.xlsx

Contenu généré par generate_excel.py :

```
policyID  statecode  county
1         FL        A
1         FL        A      ← doublon
None      TX        B      ← policyID manquant
2         None      C      ← statecode manquant
```

```
from pathlib import Path
import pandas as pd
```

```
inp  = Path("data") / "q20_pipeline.xlsx"
outp = Path("data") / "q20_pipeline_clean.xlsx"
```

```
COLONNES_REQUISES = {"policyID", "statecode", "county"}
```

```
# — Étape 1 : vérifier l'existence du fichier
if not inp.exists():
    raise FileNotFoundError(f"Fichier input introuvable : {inp}")
```

```
# — Étape 2 : valider le schéma (sans charger les données)
df_header = pd.read_excel(inp, nrows=0)
manquantes = COLONNES_REQUISES - set(df_header.columns)
```

```

if manquantes:
    raise ValueError(f"Schéma invalide : {sorted(manquantes)}")
print("✅ Schéma valide")

# — Étape 3 : charger Les données —————
df = pd.read_excel(inp)
print(f"\n✅ Données brutes ({len(df)} lignes) :")
print(df.to_string(index=False))

# — Étape 4 : audit avant nettoyage —————
print(f"\n✅ Audit :")
print(f"    Doublons complets : {df.duplicated().sum()}")
print(f"    policyID manquants : {df['policyID'].isna().sum()}")
print(f"    statecode manquants: {df['statecode'].isna().sum()}")

avant = len(df)

# — Étape 5 : nettoyage —————
df = df.dropna(subset=["policyID"]) # supprimer si policyID manquant
df = df.drop_duplicates()         # supprimer les doublons exacts

apres = len(df)
print(f"\n✅ Nettoyage : {avant} → {apres} lignes ({avant - apres} supprimées)")

# — Étape 6 : export —————
df.to_excel(outp, index=False) # index=False : pas de colonne 0,1,2...
print(f"\n✅ Export : {outp}")

# — Étape 7 : contrôle de la sortie —————
check = pd.read_excel(outp)
print(f"\n✅ Contrôle ({check.shape[0]} lignes, {check.shape[1]} colonnes) :")
print(check.to_string(index=False))

✅ Schéma valide

✅ Données brutes (4 lignes) :
policyID statecode county
1.0      FL      A
1.0      FL      A
NaN      TX      B
2.0      None     C

✅ Audit :
    Doublons complets : 1
    policyID manquants : 1
    statecode manquants: 1

```

Nettoyage : 4 → 2 lignes (2 supprimées)

Export : data/q20_pipeline_clean.xlsx

Contrôle (2 lignes, 3 colonnes) :

policyID	statecode	county
1.0	FL	A
2.0	None	C

- **to_excel(outp, index=False)** : **index=False est indispensable** — sans ça, pandas ajoute une colonne 0, 1, 2... parasite comme première colonne
- **dropna(subset=["policyID"])** : supprime uniquement les lignes où policyID est NaN —les autres colonnes manquantes sont tolérées
- **drop_duplicates()** : supprime les lignes exactement identiques sur toutes les colonnes
- **toujours relire après export** : `pd.read_excel(outp)` confirme que le fichier écrit est correct —filet de sécurité indispensable en production

□ Récapitulatif —les 7 points clés

load_workbook(read_only=True) pour les gros fichiers ·**max_row peut surestimer** —toujours compter les not None ·**nrows=0** pour valider le schéma sans charger les données ·**usecols** pour ne lire que les colonnes utiles ·**is None** (openpyxl) ou **isna()** (pandas) pour les cellules vides ·**to_excel(index=False)** obligatoire à l'export ·**valider schéma** → **nettoyer** → **exporter** → **contrôler** : l'ordre d'un pipeline robuste

20 questions intelligentes –Excel avec Python (avec fichiers input par cas)

1) Pourquoi xlrd renvoie 119736.0 au lieu de 119736

Fichier input

Créer le fichier **q01_make_input.py** :

```
"""
```

```
Génère un fichier Excel où un identifiant ressemble à un entier,  
mais Excel le stocke comme nombre (souvent traité comme float selon la lib).  
"""
```



```

val_int = int(val)
print("Conversion int :", val_int)
print("Type après conversion :", type(val_int))

```

2) Pourquoi openpyxl max_row peut surestimer

Fichier input

Créer le fichier **q02_make_input.py** :

```

"""
Génère un fichier Excel avec une dernière ligne "touchée"
puis vidée, ce qui peut laisser max_row élevé.
"""

from pathlib import Path
from openpyxl import Workbook

out_dir = Path("data")
out_dir.mkdir(exist_ok=True)

wb = Workbook()
ws = wb.active

ws["A1"] = "col1"
ws["A2"] = "ok"
ws["A100"] = "temp"
ws["A100"] = None

xlsx_path = out_dir / "q02_maxrow.xlsx"
wb.save(xlsx_path)

print("Créé :", xlsx_path)

```

Démo

Créer le fichier **q02_demo.py** :

```

"""
But :
- Afficher max_row
- Vérifier si la dernière ligne est réellement vide

À exécuter :
- python q02_make_input.py

```



```

- python q02_demo.py
"""

from pathlib import Path
from openpyxl import load_workbook

xlsx_path = Path("data") / "q02_maxrow.xlsx"

wb = load_workbook(xlsx_path)
ws = wb.active

mr = ws.max_row
print("max_row :", mr)

last_values = [c.value for c in ws[mr]]
print("Valeurs dernière ligne :", last_values)

if all(v is None for v in last_values):
    print("Dernière ligne réellement vide")
else:
    print("Dernière ligne contient des données")

```

3) Lire une cellule vs lire une plage : quel coût

Fichier input

Créer le fichier **q03_make_input.py** :

```

"""
Génère un tableau 5000 lignes x 10 colonnes pour tester la lecture.
"""

from pathlib import Path
from openpyxl import Workbook

out_dir = Path("data")
out_dir.mkdir(exist_ok=True)

wb = Workbook()
ws = wb.active

for c in range(1, 11):
    ws.cell(row=1, column=c).value = f"h{c}"

for r in range(2, 5002):

```

Démo

///

/// /// ///

4) Pourquoi read_only=True change tout en openpyxl

Fichier input

Créer le fichier **q04_make_input.py** :

```
"""
Génère un fichier volumineux pour comparer lecture normale vs read_only.
"""

from pathlib import Path
from openpyxl import Workbook

out_dir = Path("data")
out_dir.mkdir(exist_ok=True)

wb = Workbook()
ws = wb.active
ws.append(["id", "statecode", "amount"])

for i in range(1, 30001):
    ws.append([i, "FL" if i % 2 == 0 else "TX", i * 0.5])

xlsx_path = out_dir / "q04_readonly.xlsx"
wb.save(xlsx_path)

print("Créé :", xlsx_path)
```

Démo

Créer le fichier **q04_demo.py** :

```
"""
But :
- Comparer le temps de parcours des lignes
- Montrer iter_rows(values_only=True) en read_only

À exécuter :
- python q04_make_input.py
- python q04_demo.py
"""

from pathlib import Path
import time
from openpyxl import load_workbook

xlsx_path = Path("data") / "q04_readonly.xlsx"
```

```

t0 = time.time()
wb1 = load_workbook(xlsx_path, read_only=False)
ws1 = wb1.active
s = 0
for row in ws1.iter_rows(min_row=2, values_only=True):
    s += row[0]
t1 = time.time()

t2 = time.time()
wb2 = load_workbook(xlsx_path, read_only=True)
ws2 = wb2.active
s2 = 0
for row in ws2.iter_rows(min_row=2, values_only=True):
    s2 += row[0]
t3 = time.time()

print("Somme contrôle :", s, s2)
print("Normal (s) :", round(t1 - t0, 4))
print("Read_only (s) :", round(t3 - t2, 4))

```

5) Pourquoi pandas peut convertir un identifiant en float

Fichier input

Créer le fichier **q05_make_input.py** :

```

"""
Génère un Excel avec identifiants et une valeur manquante,
puis pandas peut basculer la colonne en float.
"""

from pathlib import Path
from openpyxl import Workbook

out_dir = Path("data")
out_dir.mkdir(exist_ok=True)

wb = Workbook()
ws = wb.active
ws.append(["policyID", "statecode"])
ws.append([119736, "FL"])
ws.append([None, "FL"])
ws.append([448094, "TX"])

```

```
xlsx_path = out_dir / "q05_dtype.xlsx"
wb.save(xlsx_path)
```

```
print("Créé :", xlsx_path)
```

Démo

Créer le fichier **q05_demo.py** :

```
"""
But :
- Montrer Le type inféré par pandas
- Forcer policyID en texte pour éviter float

À exécuter :
- python q05_make_input.py
- python q05_demo.py
"""
```

```
from pathlib import Path
import pandas as pd
```

```
xlsx_path = Path("data") / "q05_dtype.xlsx"
```

```
df_auto = pd.read_excel(xlsx_path)
print("Types auto :")
print(df_auto.dtypes)
print(df_auto)
```

```
df_str = pd.read_excel(xlsx_path, dtype={"policyID": "string"})
print("Types forcés :")
print(df_str.dtypes)
print(df_str)
```

6) usecols : réduire mémoire et temps

Fichier input

Créer le fichier **q06_make_input.py** :

```
"""
Génère un Excel avec 20 colonnes, mais on n'en lira que 2.
"""
```

```
from pathlib import Path
```



```

print("All shape :", df_all.shape, "temps(s) :", round(t1 - t0, 4))
print("2 cols shape :", df_2.shape, "temps(s) :", round(t3 - t2, 4))
print(df_2.head())

```

7) nrows : lire un échantillon pour valider rapidement

Fichier input

Créer le fichier **q07_make_input.py** :

```

"""
Génère un Excel de 50 000 lignes pour tester nrows.
"""

```

```

from pathlib import Path
from openpyxl import Workbook

out_dir = Path("data")
out_dir.mkdir(exist_ok=True)

wb = Workbook()
ws = wb.active
ws.append(["id", "value"])

for i in range(1, 50001):
    ws.append([i, f"v{i}"])

xlsx_path = out_dir / "q07_nrows.xlsx"
wb.save(xlsx_path)

print("Créé :", xlsx_path)

```

Démo

Créer le fichier **q07_demo.py** :

```

"""
But :
- Charger 10 lignes seulement pour validation
- Puis charger tout si nécessaire

À exécuter :
- python q07_make_input.py
- python q07_demo.py

```

```

"""

from pathlib import Path
import pandas as pd

xlsx_path = Path("data") / "q07_nrows.xlsx"

df_sample = pd.read_excel(xlsx_path, nrows=10)
print("Échantillon :")
print(df_sample)

df_full = pd.read_excel(xlsx_path)
print("Total lignes :", len(df_full))

```

8) Doublons : duplicated sur une clé métier

Fichier input

Créer le fichier **q08_make_input.py** :

```

"""
Génère un Excel avec des policyID dupliqués.
"""

from pathlib import Path
from openpyxl import Workbook

out_dir = Path("data")
out_dir.mkdir(exist_ok=True)

wb = Workbook()
ws = wb.active
ws.append(["policyID", "statecode"])
ws.append([100, "FL"])
ws.append([101, "TX"])
ws.append([100, "FL"])
ws.append([102, "TX"])
ws.append([101, "TX"])

xlsx_path = out_dir / "q08_dups.xlsx"
wb.save(xlsx_path)

print("Créé :", xlsx_path)

```


Démo

Créer le fichier **q08_demo.py** :

```
"""
But :
- Identifier les doublons sur policyID
- Afficher les lignes concernées

À exécuter :
- python q08_make_input.py
- python q08_demo.py
"""

from pathlib import Path
import pandas as pd

xlsx_path = Path("data") / "q08_dups.xlsx"

df = pd.read_excel(xlsx_path)
mask = df.duplicated(subset=["policyID"], keep=False)
dups = df[mask].sort_values("policyID")

print("Doublons trouvés :", len(dups))
print(dups)
```

9) Lignes vides : comment les repérer proprement

Fichier input

Créer le fichier **q09_make_input.py** :

```
"""
Génère un Excel avec une ligne complètement vide et une ligne partiellement vide.
"""

from pathlib import Path
from openpyxl import Workbook

out_dir = Path("data")
out_dir.mkdir(exist_ok=True)

wb = Workbook()
ws = wb.active
ws.append(["a", "b", "c"])
ws.append([1, 2, 3])
```

Démo

'''

À exécuter :

```
from pathlib import Path
import pandas as pd
```

10) Colonnes obligatoires : valider le schéma avant traitement

34

```
"""
Génère un Excel volontairement incomplet : il manque une colonne attendue.
"""
```

```
from pathlib import Path
from openpyxl import Workbook

out_dir = Path("data")
out_dir.mkdir(exist_ok=True)

wb = Workbook()
ws = wb.active
ws.append(["policyID", "statecode"]) # county manquante
ws.append([1, "FL"])

xlsx_path = out_dir / "q10_schema.xlsx"
wb.save(xlsx_path)

print("Créé :", xlsx_path)
```

Démo

Créer le fichier **q10_demo.py** :

```
"""
But :
- Exiger un schéma minimal
- Bloquer immédiatement si colonne manquante

À exécuter :
- python q10_make_input.py
- python q10_demo.py
"""
```

```
from pathlib import Path
import pandas as pd

xlsx_path = Path("data") / "q10_schema.xlsx"

required = {"policyID", "statecode", "county"}

df = pd.read_excel(xlsx_path)
missing = required - set(df.columns)

print("Colonnes présentes :", list(df.columns))
```

```

if missing:
    print("Colonnes manquantes :", sorted(missing))
    raise ValueError("Schéma invalide")

```

11) Noms de colonnes sales : espaces, doublons, casse

Fichier input

Créer le fichier **q11_make_input.py** :

```

"""
Génère un Excel avec des noms de colonnes problématiques.
"""

from pathlib import Path
from openpyxl import Workbook

out_dir = Path("data")
out_dir.mkdir(exist_ok=True)

wb = Workbook()
ws = wb.active
ws.append([" PolicyID ", "STATECODE", "statecode"])
ws.append([1, "FL", "FL"])

xlsx_path = out_dir / "q11_dirty_headers.xlsx"
wb.save(xlsx_path)

print("Créé :", xlsx_path)

```

Démo

Créer le fichier **q11_demo.py** :

```

"""
But :
- Nettoyer les en-têtes
- Rendre les noms cohérents et uniques

À exécuter :
- python q11_make_input.py
- python q11_demo.py
"""

from pathlib import Path

```

```

import pandas as pd

xlsx_path = Path("data") / "q11_dirty_headers.xlsx"
df = pd.read_excel(xlsx_path)

print("Avant :", list(df.columns))

clean = []
seen = {}
for col in df.columns:
    c = str(col).strip().lower()
    if c in seen:
        seen[c] += 1
        c = f"{c}_{seen[c]}"
    else:
        seen[c] = 0
    clean.append(c)

df.columns = clean

print("Après :", list(df.columns))
print(df.head())

```

12) Dates Excel : pourquoi ça casse, et comment stabiliser

Fichier input

Créer le fichier **q12_make_input.py** :

```

"""
Génère un Excel avec une vraie date et une date sous forme texte.
"""

```

```

from pathlib import Path
from datetime import datetime
from openpyxl import Workbook

out_dir = Path("data")
out_dir.mkdir(exist_ok=True)

wb = Workbook()
ws = wb.active
ws.append(["id", "date_ok", "date_text"])
ws.append([1, datetime(2026, 3, 1), "2026-03-01"])
ws.append([2, datetime(2026, 3, 2), "03/02/2026"])

```

```
xlsx_path = out_dir / "q12_dates.xlsx"
wb.save(xlsx_path)

print("Créé :", xlsx_path)
```

Démo

Créer le fichier **q12_demo.py** :

```
"""
```

But :

- Montrer Les types de colonnes
- Normaliser en datetime côté pandas

À exécuter :

- `python q12_make_input.py`
- `python q12_demo.py`

```
"""
```

```
from pathlib import Path
import pandas as pd
```

```
xlsx_path = Path("data") / "q12_dates.xlsx"
df = pd.read_excel(xlsx_path)
```

```
print("Types initiaux :")
print(df.dtypes)
print(df)
```

```
df["date_text_norm"] = pd.to_datetime(df["date_text"], errors="coerce", dayfirst=True)
```

```
print("Après normalisation :")
print(df[["date_ok", "date_text", "date_text_norm"]])
```

13) Valeurs numériques avec virgule : 12,5 devient texte

Fichier input

Créer le fichier **q13_make_input.py** :

```
"""
```

Génère un Excel avec un montant stocké en texte contenant une virgule.

```
"""
```

```

from pathlib import Path
from openpyxl import Workbook

out_dir = Path("data")
out_dir.mkdir(exist_ok=True)

wb = Workbook()
ws = wb.active
ws.append(["id", "amount_text"])
ws.append([1, "12,5"])
ws.append([2, "100,00"])
ws.append([3, "7,25"])

xlsx_path = out_dir / "q13_comma.xlsx"
wb.save(xlsx_path)

print("Créé :", xlsx_path)

```

Démo

Créer le fichier **q13_demo.py** :

"""

But :

- *Convertir proprement une colonne texte avec virgule en float*

À exécuter :

- *python q13_make_input.py*

- *python q13_demo.py*

"""

```

from pathlib import Path
import pandas as pd

xlsx_path = Path("data") / "q13_comma.xlsx"
df = pd.read_excel(xlsx_path)

print("Avant :")
print(df)
print(df.dtypes)

df["amount"] = (
    df["amount_text"]
    .astype(str)
    .str.replace(",", ".", regex=False)
    .astype(float)
)

```

```
)

print("Après :")
print(df)
print(df.dtypes)
```

14) Lecture multi-feuilles : dict de DataFrames

Fichier input

Créer le fichier **q14_make_input.py** :

```
"""
Génère un Excel avec deux feuilles : clients et commandes.
"""

from pathlib import Path
from openpyxl import Workbook

out_dir = Path("data")
out_dir.mkdir(exist_ok=True)

wb = Workbook()

ws1 = wb.active
ws1.title = "clients"
ws1.append(["client_id", "name"])
ws1.append([1, "A"])
ws1.append([2, "B"])

ws2 = wb.create_sheet("orders")
ws2.append(["order_id", "client_id", "amount"])
ws2.append([10, 1, 50])
ws2.append([11, 2, 70])

xlsx_path = out_dir / "q14_sheets.xlsx"
wb.save(xlsx_path)

print("Créé :", xlsx_path)
```

Démo

Créer le fichier **q14_demo.py** :


```

"""
But :
- Charger toutes les feuilles
- Accéder par nom

À exécuter :
- python q14_make_input.py
- python q14_demo.py
"""

from pathlib import Path
import pandas as pd

xlsx_path = Path("data") / "q14_sheets.xlsx"

sheets = pd.read_excel(xlsx_path, sheet_name=None)

print("Feuilles :", list(sheets.keys()))
print("Clients :")
print(sheets["clients"])

print("Orders :")
print(sheets["orders"])

```

15) Jointure métier : enrichir commandes avec clients

Fichier input

Réutilise le fichier créé par la question 14.

Démo

Créer le fichier **q15_demo.py** :

```

"""
But :
- Lire deux feuilles
- Faire une jointure client_id
- Obtenir un dataset enrichi

À exécuter :
- python q14_make_input.py
- python q15_demo.py
"""

```

```

from pathlib import Path
import pandas as pd

xlsx_path = Path("data") / "q14_sheets.xlsx"

clients = pd.read_excel(xlsx_path, sheet_name="clients")
orders = pd.read_excel(xlsx_path, sheet_name="orders")

enriched = orders.merge(clients, on="client_id", how="left")

print(enriched)

```

16) Détecter des références orphelines (order sans client)

Fichier input

Créer le fichier **q16_make_input.py** :

```

"""
Génère clients et orders, avec une commande qui référence un client inexistant.
"""

from pathlib import Path
from openpyxl import Workbook

out_dir = Path("data")
out_dir.mkdir(exist_ok=True)

wb = Workbook()

clients = wb.active
clients.title = "clients"
clients.append(["client_id", "name"])
clients.append([1, "A"])
clients.append([2, "B"])

orders = wb.create_sheet("orders")
orders.append(["order_id", "client_id", "amount"])
orders.append([10, 1, 50])
orders.append([11, 99, 70])

xlsx_path = out_dir / "q16_orphans.xlsx"
wb.save(xlsx_path)

print("Créé :", xlsx_path)

```

Démo

Créer le fichier **q16_demo.py** :

```
"""
But :
- Trouver Les commandes orphelines
- Afficher Les lignes problématiques

À exécuter :
- python q16_make_input.py
- python q16_demo.py
"""

from pathlib import Path
import pandas as pd

xlsx_path = Path("data") / "q16_orphans.xlsx"

clients = pd.read_excel(xlsx_path, sheet_name="clients")
orders = pd.read_excel(xlsx_path, sheet_name="orders")

enriched = orders.merge(clients, on="client_id", how="left")

orphans = enriched[enriched["name"].isna()]

print("Orphelines :", len(orphans))
print(orphans)
```

17) Gérer un fichier verrouillé ou inaccessible : stratégie de fallback

Fichier input

Créer le fichier **q17_make_input.py** :

```
"""
Génère un Excel standard à Lire.
"""

from pathlib import Path
from openpyxl import Workbook

out_dir = Path("data")
out_dir.mkdir(exist_ok=True)
```


18) Export propre : éviter la colonne index parasite

Fichier input

Créer le fichier **q18_make_input.py** :

```
"""
Génère un Excel simple à exporter après transformation.
"""

from pathlib import Path
from openpyxl import Workbook

out_dir = Path("data")
out_dir.mkdir(exist_ok=True)

wb = Workbook()
ws = wb.active
ws.append(["policyID", "statecode"])
ws.append([1, "FL"])
ws.append([2, "TX"])

xlsx_path = out_dir / "q18_export_in.xlsx"
wb.save(xlsx_path)

print("Créé :", xlsx_path)
```

Démo

Créer le fichier **q18_demo.py** :

```
"""
But :
- Transformer
- Exporter sans index
- Relire pour vérifier

À exécuter :
- python q18_make_input.py
- python q18_demo.py
"""

from pathlib import Path
import pandas as pd

inp = Path("data") / "q18_export_in.xlsx"
outp = Path("data") / "q18_export_out.xlsx"
```

```

df = pd.read_excel(inp)
df["statecode"] = df["statecode"].str.lower()

df.to_excel(outp, index=False)

check = pd.read_excel(outp)

print("Sortie relue :", check.shape)
print(check)

```

19) Comparer deux versions d'un Excel : détecter les écarts

Fichier input

Créer le fichier **q19_make_input.py** :

```

"""
Génère deux fichiers Excel proches, avec une différence à détecter.
"""

```

```

from pathlib import Path
from openpyxl import Workbook

```

```

out_dir = Path("data")
out_dir.mkdir(exist_ok=True)

```

```

def make_file(p, rows):
    wb = Workbook()
    ws = wb.active
    ws.append(["id", "value"])
    for r in rows:
        ws.append(r)
    wb.save(p)
    print("Créé :", p)

```

```

make_file(out_dir / "q19_v1.xlsx", [(1, "A"), (2, "B"), (3, "C")])
make_file(out_dir / "q19_v2.xlsx", [(1, "A"), (2, "B2"), (4, "D")])

```

Démo

Créer le fichier **q19_demo.py** :

```

"""
But :

```

- Comparer v1 et v2 sur la clé id
- Afficher ajoutés, supprimés, modifiés

À exécuter :

- `python q19_make_input.py`
 - `python q19_demo.py`
- """

```
from pathlib import Path
import pandas as pd

v1 = pd.read_excel(Path("data") / "q19_v1.xlsx")
v2 = pd.read_excel(Path("data") / "q19_v2.xlsx")

m1 = v1.set_index("id")
m2 = v2.set_index("id")

added = m2.loc[m2.index.difference(m1.index)]
removed = m1.loc[m1.index.difference(m2.index)]

common = m1.index.intersection(m2.index)
changed = common[m1.loc[common, "value"] != m2.loc[common, "value"]]

print("Ajoutés :")
print(added)

print("Supprimés :")
print(removed)

print("Modifiés :")
for i in changed:
    print("id", i, ":", m1.loc[i, "value"], "->", m2.loc[i, "value"])
```

20) Pipeline reproductible : validation + nettoyage + export + contrôle

Fichier input

Créer le fichier `q20_make_input.py` :

"""

Génère un Excel contenant :

- doublons
- valeurs manquantes
- colonnes attendues

```

"""

from pathlib import Path
from openpyxl import Workbook

out_dir = Path("data")
out_dir.mkdir(exist_ok=True)

wb = Workbook()
ws = wb.active
ws.append(["policyID", "statecode", "county"])
ws.append([1, "FL", "A"])
ws.append([1, "FL", "A"])      # doublon
ws.append([None, "TX", "B"])  # policyID manquant
ws.append([2, None, "C"])     # statecode manquant

xlsx_path = out_dir / "q20_pipeline.xlsx"
wb.save(xlsx_path)

print("Créé :", xlsx_path)

```

Démo

Créer le fichier **q20_demo.py** :

```

"""
But :
- Vérifier existence fichier
- Valider schéma
- Nettoyer (policyID non nul, doublons)
- Exporter
- Relire pour contrôler

```

```

À exécuter :
- python q20_make_input.py
- python q20_demo.py
"""

```

```

from pathlib import Path
import pandas as pd

inp = Path("data") / "q20_pipeline.xlsx"
outp = Path("data") / "q20_pipeline_clean.xlsx"

required = {"policyID", "statecode", "county"}

```



```

if not inp.exists():
    raise FileNotFoundError("Fichier input introuvable")

df = pd.read_excel(inp)

missing = required - set(df.columns)
if missing:
    raise ValueError("Schéma invalide, colonnes manquantes : " + ", ".join(sorted(miss

before = len(df)

df = df.dropna(subset=["policyID"])
df = df.drop_duplicates()

after = len(df)

df.to_excel(outp, index=False)

check = pd.read_excel(outp)

print("Avant nettoyage :", before)
print("Après nettoyage :", after)
print("Sortie :", outp)
print(check)

```